

Web Dev Quiz -- Fill in the Blank

Vsego voprosov: 86 | Endterm + Midterm

#138

Endterm

Fill in the Blank

Model field types

Complete the Django model definition.

```
from django.db import models

class Product(models.[____1____]):
    name = models.CharField([____2____]=300)
    price = models.[____3____](max_digits=10, decimal_places=2)
```

Answers:

- [1] Model
- [2] max_length
- [3] DecimalField

#139

Endterm

Fill in the Blank

ForeignKey declaration

Fill in the foreign-key relation between Product and Category.

```
category = models.[____1____](
    Category,
    [____2____]=models.CASCADE,
    related_name='[____3____]',
)
```

Answers:

- [1] ForeignKey
- [2] on_delete
- [3] products

#140

Endterm

Fill in the Blank

Model __str__

Define the human-readable string for a Category instance.

```
class Category(models.Model):
    name = models.CharField(max_length=300)

    def [____1____](self):
        return self.[____2____]
```

Answers:

- [1] __str__
- [2] name

#141

Endterm

Fill in the Blank

DRF function-based view

Complete the DRF function-based list/create view.

```
@[____1____](['GET', 'POST'])
def products_list(request):
    if request.method == 'GET':
        products = Product.objects.all()
```

```
serializer = ProductSerializer(products, [____2____]=True)
return [____3____](serializer.data)
```

Answers:

- [1] api_view
- [2] many
- [3] Response

#142

Endterm

Fill in the Blank

DRF APIView class

Fill in the class-based list+create view.

```
from rest_framework.views import APIView

class ProductListAPIView([____1____]):
    def [____2____](self, request):
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    def [____3____](self, request):
        serializer = ProductSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```

Answers:

- [1] APIView
- [2] get
- [3] post

#143

Endterm

Fill in the Blank

urls.py with as_view()

Wire up two class-based views in urls.py.

```
urlpatterns = [
    path('products/', [____1____].as_view()),
    path('products/<int:[____2____]>/', ProductDetailAPIView.[____3____]()),
]
```

Answers:

- [1] ProductListAPIView
- [2] pk
- [3] as_view

#144

Endterm

Fill in the Blank

Serializer save flow

Validate, save, and return the serialized object.

```
serializer = ProductSerializer(data=request.data)
if serializer.[____1____]():
    serializer.[____2____]()
    return Response(serializer.[____3____], status=status.HTTP_201_CREATED)
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Answers:

```
[1] is_valid
[2] save
[3] data
```

#145

Endterm

Fill in the Blank

GenericAPIView with mixins

Fill in the mixin combination for a list+create view.

```
class ProductListAPIView(mixins.[____1____],
                        mixins.CreateModelMixin,
                        generics.[____2____]):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    def get(self, request, *args, **kwargs):
        return self.[____3____](request, *args, **kwargs)
```

Answers:

```
[1] ListModelMixin
[2] GenericAPIView
[3] list
```

#146

Endterm

Fill in the Blank

Concrete generic views

Use the concrete generic views for full CRUD.

```
class ProductListAPIView(generics.[____1____]):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

class ProductDetailAPIView(generics.[____2____]):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    [____3____] = 'product_id'
```

Answers:

```
[1] ListCreateAPIView
[2] RetrieveUpdateDestroyAPIView
[3] lookup_url_kwarg
```

#147

Endterm

Fill in the Blank

ModelSerializer Meta

Auto-generate fields from the Product model.

```
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.[____1____]):
    class [____2____]:
        model = Product
        [____3____] = '__all__'
```

Answers:

```
[1] ModelSerializer
[2] Meta
```

[3] fields

#148

Endterm

Fill in the Blank

DELETE returns 204

Implement the destroy handler returning the right status.

```
def delete(self, request, product_id):
    product = self.get_object(product_id)
    product.[____1____]()
    return Response(status=status.HTTP_204_[____2____])
```

Answers:

[1] delete

[2] NO_CONTENT

#149

Endterm

Fill in the Blank

Simple JWT URLs

Wire up the obtain-pair and refresh endpoints.

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path('api/token/', [____1____].as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', [____2____].as_view(), name='token_refresh'),
]
```

Answers:

[1] TokenObtainPairView

[2] TokenRefreshView

#150

Endterm

Fill in the Blank

DRF settings dict

Configure pagination and the filter backend in settings.py.

```
REST_FRAMEWORK = {
    '[____1____]': 'rest_framework.pagination.PageNumberPagination',
    '[____2____]': 10,
    'DEFAULT_FILTER_BACKENDS': (
        'django_filters.rest_framework.[____3____]',
    ),
}
```

Answers:

[1] DEFAULT_PAGINATION_CLASS

[2] PAGE_SIZE

[3] DjangoFilterBackend

#151

Endterm

Fill in the Blank

Search and ordering filters

Enable search and ordering on a list view.

```
from rest_framework import filters, generics

class ProductListAPIView(generics.ListAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

```
filter_backends = (filters.[____1____], filters.[____2____])
search_fields = ('name',)
[____3____] = ('name',)
```

Answers:

- [1] SearchFilter
- [2] OrderingFilter
- [3] ordering_fields

#152

Endterm

Fill in the Blank

JWT auth + permission settings

Make JWTAuthentication the default and require login.

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.[____1____]',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.[____2____]',
    ),
}
```

Answers:

- [1] JWTAuthentication
- [2] IsAuthenticated

#153

Endterm

Fill in the Blank

JWT request header

Show the HTTP header that carries an access token to a protected endpoint.

```
GET /api/products/ HTTP/1.1
Host: example.com
[____1____]: [____2____] eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Answers:

- [1] Authorization
- [2] Bearer

#154

Endterm

Fill in the Blank

ViewSet and router

Register a full-CRUD ViewSet with the default router.

```
from rest_framework import viewsets, routers

class ProductViewSet(viewsets.[____1____]):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

router = routers.[____2____]()
router.[____3____](r'products', ProductViewSet)
```

Answers:

- [1] ModelViewSet
 - [2] DefaultRouter
 - [3] register
-

Custom ViewSet endpoint

Add a custom list-style endpoint to a ViewSet.

```
from rest_framework.decorators import action
from rest_framework.response import Response

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    @[____1____](detail=[____2____], methods=['get'])
    def featured(self, request):
        qs = Product.objects.filter(is_featured=True)
        serializer = self.get_serializer(qs, [____3____]=True)
        return Response(serializer.data)
```

Answers:

[1] action

[2] False

[3] many

DRF status constants

Pick the correct HTTP status constant in each line.

```
from rest_framework import status

# successful create
status.HTTP_201_[____1____]

# validation error
status.HTTP_[____2____]_BAD_REQUEST

# missing object
status.HTTP_[____3____]_NOT_FOUND
```

Answers:

[1] CREATED

[2] 400

[3] 404

APIView get_object pattern

Implement the helper that fetches a Product or returns None.

```
class ProductDetailAPIView(APIView):
    def get_object(self, product_id):
        try:
            return Product.objects.[____1____](pk=product_id)
        except Product.[____2____]:
            return None

    def get(self, request, product_id):
        product = self.[____3____](product_id)
        if product is None:
```

```
        return Response(status=status.HTTP_404_NOT_FOUND)
    serializer = ProductSerializer(product)
    return Response(serializer.data)
```

Answers:

- [1] get
- [2] DoesNotExist
- [3] get_object

#158

Endterm

Fill in the Blank

Custom Serializer + create

Define a non-Model serializer with explicit fields and a create() method.

```
class ProductSerializer(serializers.[____1____]):
    id = serializers.IntegerField(read_only=True)
    name = serializers.[____2____](max_length=300)
    price = serializers.DecimalField(max_digits=10, decimal_places=2)

    def [____3____](self, validated_data):
        return Product.objects.create(**validated_data)
```

Answers:

- [1] Serializer
- [2] CharField
- [3] create

#159

Endterm

Fill in the Blank

Serializer update method

Implement update() so the serializer can write back to an existing instance.

```
def update(self, [____1____], validated_data):
    instance.name = validated_data.get('name', instance.name)
    instance.price = validated_data.get('price', instance.price)
    instance.[____2____]()
    return [____3____]
```

Answers:

- [1] instance
- [2] save
- [3] instance

#160

Endterm

Fill in the Blank

validate_<field> method

Reject negative prices on the price field.

```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = '__all__'

    def [____1____]_price(self, value):
        if value < 0:
            raise serializers.[____2____]('Price must be non-negative')
        return [____3____]
```

Answers:

```
[1] validate
[2] ValidationError
[3] value
```

#161

Endterm

Fill in the Blank

Cross-field validate()

Reject input where end is not after start.

```
class TimeRangeSerializer(serializers.Serializer):
    start = serializers.IntegerField()
    end = serializers.IntegerField()

    def [____1____](self, attrs):
        if attrs['start'] >= attrs['end']:
            raise serializers.[____2____]('start must be before end')
        return [____3____]
```

Answers:

```
[1] validate
[2] ValidationError
[3] attrs
```

#162

Endterm

Fill in the Blank

SerializerMethodField

Expose a computed discounted_price field on the serializer.

```
class ProductSerializer(serializers.ModelSerializer):
    discounted_price = serializers.[____1____]()

    class Meta:
        model = Product
        fields = ['id', 'name', 'price', 'discounted_price']

    def [____2____]_discounted_price(self, [____3____]):
        return obj.price * Decimal('0.9')
```

Answers:

```
[1] SerializerMethodField
[2] get
[3] obj
```

#163

Endterm

Fill in the Blank

Nested serializer

Embed Category data inside Product output.

```
class CategorySerializer(serializers.[____1____]):
    class Meta:
        model = Category
        fields = ['id', 'name']

class ProductSerializer(serializers.ModelSerializer):
    category = [____2____]([____3____]=True)

    class Meta:
        model = Product
```



```
fields = ['id', 'name', 'category']
```

Answers:

- [1] ModelSerializer
- [2] CategorySerializer
- [3] read_only

#164

Endterm

Fill in the Blank

write_only and read_only_fields

Hide the password from responses while keeping id read-only.

```
class UserSerializer(serializers.ModelSerializer):
    password = serializers.CharField([__1__]=True)

    class Meta:
        model = User
        fields = ['id', 'username', 'password']
        [__2__]_fields = ['id']

    def create(self, validated_data):
        return User.objects.[__3__](**validated_data)
```

Answers:

- [1] write_only
- [2] read_only
- [3] create

#165

Endterm

Fill in the Blank

perform_create injects user

Attach the request user as the owner during create.

```
class ProductCreateView(generics.CreateAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]

    def [__1__](self, serializer):
        serializer.save([__2__]=self.[__3__].user)
```

Answers:

- [1] perform_create
- [2] owner
- [3] request

#166

Endterm

Fill in the Blank

get_queryset filtered by user

Only return orders that belong to the current user.

```
class MyOrdersAPIView(generics.ListAPIView):
    serializer_class = OrderSerializer
    permission_classes = [IsAuthenticated]

    def [__1__](self):
        return Order.objects.[__2__](user=self.request.[__3__])
```

Answers:

```
[1] get_queryset
[2] filter
[3] user
```

#167

Endterm

Fill in the Blank

get_serializer_class per action

Return different serializers for read vs write actions on a ViewSet.

```
class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.[__1__]()

    def [__2__](self):
        if self.action in ['list', '[__3__]']:
            return ProductReadSerializer
        return ProductWriteSerializer
```

Answers:

```
[1] all
[2] get_serializer_class
[3] retrieve
```

#168

Endterm

Fill in the Blank

JWT auth + permission settings (2)

Make Simple JWT the default auth and require login by default.

```
REST_FRAMEWORK = {
    '[__1__]': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    '[__2__]': (
        'rest_framework.permissions.[__3__]',
    ),
}
```

Answers:

```
[1] DEFAULT_AUTHENTICATION_CLASSES
[2] DEFAULT_PERMISSION_CLASSES
[3] IsAuthenticated
```

#169

Endterm

Fill in the Blank

JWT URLs imports

Import and wire the obtain-pair and refresh views.

```
from rest_framework_simplejwt.views import (
    [__1__],
    TokenRefreshView,
)

urlpatterns = [
    path('api/token/', [__1__].[__2__](), name='token_obtain_pair'),
    path('api/token/refresh/', [__3__].as_view(), name='token_refresh'),
]
```

Answers:

```
[1] TokenObtainPairView
```

```
[2] as_view
[3] TokenRefreshView
```

#170

Endterm

Fill in the Blank

Custom PageNumberPagination

Subclass the page-number pagination and wire it onto the view.

```
from rest_framework.[____1____] import PageNumberPagination

class StandardResultsPagination([____2____]):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 100

class ProductListAPIView(generics.ListAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    [____3____] = StandardResultsPagination
```

Answers:

```
[1] pagination
[2] PageNumberPagination
[3] pagination_class
```

#171

Endterm

Fill in the Blank

Custom FilterSet

Define a FilterSet for Product and attach it to the view.

```
import django_filters
from .models import Product

class ProductFilter(django_filters.[____1____]):
    min_price = django_filters.NumberFilter(field_name='price', lookup_expr='gte')
    max_price = django_filters.NumberFilter(field_name='price', lookup_expr='lte')

    class Meta:
        [____2____] = Product
        fields = ['category']

class ProductListAPIView(generics.ListAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = (DjangoFilterBackend,)
    [____3____] = ProductFilter
```

Answers:

```
[1] FilterSet
[2] model
[3] filterset_class
```

#172

Endterm

Fill in the Blank

Throttle settings

Limit anonymous and authenticated request rates globally.

```
REST_FRAMEWORK = {
```

```

    'DEFAULT_THROTTLE_CLASSES': (
        'rest_framework.throttling.[____1____]',
        'rest_framework.throttling.[____2____]',
    ),
    'DEFAULT_THROTTLE_RATES': {
        '[____3____]': '100/day',
        'user': '1000/day',
    },
}

```

Answers:

```

[1] AnonRateThrottle
[2] UserRateThrottle
[3] anon

```

#173

Endterm

Fill in the Blank

FBV with permissions

Restrict an `@api_view` function to logged-in users.

```

from rest_framework.decorators import api_view, [____1____]
from rest_framework.permissions import IsAuthenticated

@api_view(['GET'])
@[____1____]([____2____])
def my_profile(request):
    user = request.[____3____]
    return Response({'username': user.username})

```

Answers:

```

[1] permission_classes
[2] IsAuthenticated
[3] user

```

#174

Endterm

Fill in the Blank

APIView with permission_classes

Protect a class-based view and check authentication.

```

from rest_framework.permissions import [____1____]
from rest_framework import status

class ProtectedView(APIView):
    [____2____] = [____1____]

    def get(self, request):
        if not request.user.[____3____]:
            return Response(status=status.HTTP_401_UNAUTHORIZED)
        return Response({'username': request.user.username})

```

Answers:

```

[1] IsAuthenticated
[2] permission_classes
[3] is_authenticated

```

#175

Endterm

Fill in the Blank

Router URL inclusion

Mount the router-generated URLs and register two ViewSets.

```
from django.urls import path, [____1____]
from rest_framework import routers
from .views import ProductViewSet, CategoryViewSet

router = routers.[____2____]()
router.register(r'products', ProductViewSet)
router.register(r'categories', [____3____])

urlpatterns = [
    path('', include(router.urls)),
]
```

Answers:

```
[1] include
[2] DefaultRouter
[3] CategoryViewSet
```

#197

Endterm

Fill in the Blank

urls.py with admin and re_path

Wire the admin and a regex URL pattern in the project urls.py.

```
from django.[____1____] import admin
from django.urls import path, [____2____]

urlpatterns = [
    path('admin/', admin.[____3____].urls),
    [____2____](r'^time/plus/\d+/$', hours_ahead),
]
```

Answers:

```
[1] contrib
[2] re_path
[3] site
```

#198

Endterm

Fill in the Blank

HttpResponse view function

Recreate a simple 'hello' view that returns an HttpResponse.

```
from django.[____1____] import HttpResponse

def hello(request):
    return [____2____]('<h1>Hello, world!</h1>')

urlpatterns = [
    path('[____3____]', hello),
]
```

Answers:

```
[1] http
[2] HttpResponse
[3] hello/
```

#200

Endterm

Fill in the Blank

Pretty URL with capture group

Reproduce the hours_ahed pretty-URL pattern with a captured group.

```
from django.urls import [____1____]
from django.http import HttpResponse

def hours_ahed([____2____], hour):
    return HttpResponse(f'hours ahead: {hour}')
```

```
urlpatterns = [
    [____1____](r'^time/plus/(\\d+)/[____3____]', hours_ahed),
]
```

Answers:

```
[1] re_path
[2] request
[3] $
```

#201

Endterm

Fill in the Blank

render() with a template

Use the Django template system to render an HTML response.

```
from django.[____1____] import render

def home(request):
    context = {'title': 'Welcome'}
    return [____2____](request, '[____3____]', context)
```

Answers:

```
[1] shortcuts
[2] render
[3] home.html
```

#202

Endterm

Fill in the Blank

Category model with Meta

Mirror the Category class structure with verbose names in Meta.

```
from django.db import [____1____]

class Category([____1____].Model):
    name = [____1____].CharField(max_length=300)

    class [____2____]:
        verbose_name = 'Category'
        verbose_name_plural = '[____3____]'
```

```
    def __str__(self):
        return f'{self.id} - {self.name}'
```

Answers:

```
[1] models
[2] Meta
[3] Categories
```

#203

Endterm

Fill in the Blank

JWT authentication settings

Configure Simple JWT in INSTALLED_APPS and DEFAULT_AUTHENTICATION_CLASSES.

```
INSTALLED_APPS = [  
    'rest_framework',  
    '____1____',  
]  
  
REST_FRAMEWORK = {  
    '____2____': (  
        'rest_framework_simplejwt.authentication.____3____',  
    ),  
}
```

Answers:

- [1] rest_framework_simplejwt
- [2] DEFAULT_AUTHENTICATION_CLASSES
- [3] JWTAuthentication

#204

Endterm

Fill in the Blank

DjangoFilterBackend with filterset_class

Wire DjangoFilterBackend and a custom FilterSet on a list view.

```
from django_filters.rest_framework import ____1____  
from rest_framework.permissions import IsAuthenticated  
from rest_framework import generics  
  
class ProductListAPIView(generics.ListAPIView):  
    queryset = Product.objects.all()  
    serializer_class = ProductSerializer  
    filter_backends = (____1____,)  
    ____2____ = ProductFilter  
    ____3____ = (IsAuthenticated,)
```

Answers:

- [1] DjangoFilterBackend
- [2] filterset_class
- [3] permission_classes

#205

Endterm

Fill in the Blank

Mount api urls and JWT login

Add a sub-app urls.py and a Simple JWT login endpoint.

```
from django.contrib import admin  
from django.urls import path, ____1____  
from rest_framework_simplejwt.views import ____2____  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('api/', ____1____('api.urls')),  
    path('login/', ____2____.____3____()),  
]
```

Answers:

- [1] include
 - [2] TokenObtainPairView
 - [3] as_view
-

Level 0: raw Django POST handler

Recreate the manual POST handler used in the raw-Django 'Level 0' code (no DRF yet).

```
from django.views.decorators.csrf import [____1____]
from django.http import JsonResponse
import json

@[____1____]
def products_list(request):
    if request.method == 'POST':
        data = [____2____].loads(request.[____3____])
        product = Product.objects.create(**data)
        return JsonResponse(product.to_json(), status=201)
```

Answers:

```
[1] csrf_exempt
[2] json
[3] body
```

Level 2: DRF FBV GET branch

Fill in the GET branch of a DRF function-based view.

```
@api_view(['GET', 'POST'])
def products_list(request):
    if request.method == 'GET':
        products = Product.objects.[____1____]()
        serializer = ProductSerializer(products, [____2____]=True)
        return Response(serializer.[____3____])
```

Answers:

```
[1] all
[2] many
[3] data
```

Level 4: ListCreate via mixins

Combine the list and create mixins on a GenericAPIView.

```
from rest_framework import mixins, [____1____]

class ProductListAPIView(mixins.ListModelMixin,
                        mixins.[____2____],
                        [____1____].GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    def post(self, request, *args, **kwargs):
        return self.[____3____](request, *args, **kwargs)
```

Answers:

```
[1] generics
[2] CreateModelMixin
[3] create
```


Concrete RetrieveAPIView

Use the GET-only concrete generic view.

```
from rest_framework import generics

class ProductDetailAPIView(generics.[____1____]):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    [____2____] = '[____3____]'
```

Answers:

```
[1] RetrieveAPIView
[2] lookup_url_kwarg
[3] product_id
```

RetrieveUpdateDestroy via mixins

Combine the retrieve / update / destroy mixins on a detail view.

```
class ProductDetailAPIView(mixins.RetrieveModelMixin,
                           mixins.[____1____],
                           mixins.[____2____],
                           generics.GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    def delete(self, request, *args, **kwargs):
        return self.[____3____](request, *args, **kwargs)
```

Answers:

```
[1] UpdateModelMixin
[2] DestroyModelMixin
[3] destroy
```

DRF status constants — read flow

Pick the right status constants for the read-list, validation-error and not-found responses.

```
from rest_framework import status

# success on read
return Response(serializer.data, status=status.HTTP_[____1____]_OK)

# validation error
return Response(serializer.errors, status=status.HTTP_[____2____]_BAD_REQUEST)

# missing object
return Response(status=status.HTTP_[____3____]_NOT_FOUND)
```

Answers:

```
[1] 200
[2] 400
[3] 404
```

Enable CORS app and middleware

Wire django-cors-headers into INSTALLED_APPS and MIDDLEWARE.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    # ...  
    'rest_framework',  
    '____1____',  
]  
  
MIDDLEWARE = [  
    '____1____'.____2____.____3____',  
    'django.middleware.common.CommonMiddleware',  
]
```

Answers:

- [1] corsheaders
- [2] middleware
- [3] CorsMiddleware

#213

Endterm

Fill in the Blank

Pagination settings

Configure DRF pagination via the REST_FRAMEWORK dict.

```
REST_FRAMEWORK = {  
    '____1____': 'rest_framework.pagination.____2____',  
    '____3____': 10,  
}
```

Answers:

- [1] DEFAULT_PAGINATION_CLASS
- [2] PageNumberPagination
- [3] PAGE_SIZE

#214

Endterm

Fill in the Blank

SearchFilter + OrderingFilter view

Enable both the search filter and the ordering filter on a list view.

```
from rest_framework import filters, generics  
  
class ProductListAPIView(generics.ListAPIView):  
    queryset = Product.objects.all()  
    serializer_class = ProductSerializer  
    filter_backends = (filters.____1____, filters.____2____)  
    search_fields = ('name',)  
    ____3____ = ('name',)
```

Answers:

- [1] SearchFilter
- [2] OrderingFilter
- [3] ordering_fields

#215

Endterm

Fill in the Blank

Filter URL params reference

Match each DRF filter backend with its ?param=... URL parameter.

```
# DjangoFilterBackend with filterset_fields = ('name',)
http://url?[__1__]=text

# filters.SearchFilter
http://url?[__2__]=text

# filters.OrderingFilter
http://url?[__3__]=name
```

Answers:

- [1] name
- [2] search
- [3] ordering

#1253

Midterm

Fill in the Blank

Angular Component Decorator

```
import { Component } from '@angular/core';

@[__1__]({
  __2__: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'My App';
}
```

Answers:

- [1] Component
- [2] selector

#1254

Midterm

Fill in the Blank

Angular @for Loop with Track

```
<ul> @{{1}} (student of students; {{2}} student.id) { <li>{{ student.name }}</li> } </ul>
```

Answers:

- [1] for
- [2] track

#1255

Midterm

Fill in the Blank

Angular @if / @else Conditional Rendering

```
@{{1}} (isLoggedIn) { <p>Welcome back!</p> } @{{2}} { <p>Please log in.</p> }
```

Answers:

- [1] if
- [2] else

#1256

Midterm

Fill in the Blank

Two-way Data Binding with ngModel

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [__1__],
  template: `
    <input [__2__]="searchTerm" placeholder="Search...">
```

```
    })
    export class SearchComponent {
        searchTerm: string = '';
    }
}
```

Answers:

[1] FormsModule

#1257

Midterm

Fill in the Blank

Signal-based Input and Output

```
export class StudentCardComponent { name = {{1}}<string>(); delete = {{2}}<number>();
    onDelete() {
        this.delete.emit(1);
    }
}
```

Answers:

[1] input()

[2] output()

#1258

Midterm

Fill in the Blank

Component Lifecycle - ngOnInit

```
import { Component, {{1}} } from '@angular/core';
@Component({ selector: 'app-student-list', templateUrl: './student-list.html' })
export class StudentListComponent implements [____2____] {
    students: Student[] = [];

    [____3____](): void {
        this.students = this.loadStudents();
    }
}
```

Answers:

[1] OnInit

[2] OnInit

[3] ngOnInit

#1259

Midterm

Fill in the Blank

Injectable Service with providedIn

```
import { {{1}} } from '@angular/core';
@[____2____]({
    providedIn: '[____3____]'
})
export class PostsService {
    private posts: Post[] = [];
}
```

Answers:

[1] Injectable

[2] Injectable

[3] root

#1260

Midterm

Fill in the Blank

Angular Route Configuration

```
export const routes: Routes = [ { path: '', {{1}}: 'home', pathMatch: '{{2}}' }, { path: 'home', {{3}}: HomeComponent }, { path: 'posts/:id', component: PostDetailComponent }, { path: '**', component: NotFoundComponent } ];
```

Answers:

- [1] redirectTo
- [2] full
- [3] component

#1261

Midterm

Fill in the Blank

Signals and inject() for Async Data

```
export class PostsComponent implements OnInit { private postService = {{1}}(PostsService);
```

```
  posts = [____2____]<Post[]>([]);
  loading = signal(true);

  ngOnInit(): void {
    this.postService.getPosts().subscribe({
      next: (posts) => {
        this.posts.[____3____](posts);
        this.loading.set(false);
      }
    });
  }
}
```

Answers:

- [1] inject
- [2] signal
- [3] set

#1262

Midterm

Fill in the Blank

Router Outlet and Navigation Links

```
<nav> <a {{1}}="/home" {{2}}="active">Home</a> <a routerLink="/posts" routerLinkActive="active">Posts</a>
</nav>
```

```
<[____3____] />
```

Answers:

- [1] routerLink
- [2] routerLinkActive
- [3] router-outlet

#1263

Midterm

Fill in the Blank

Event binding on button

```
<button {{1}}="incClicked()">Increment</button> <p>Count: {{ count }}</p>
```

Answers:

- [1] (click)

#1264

Midterm

Fill in the Blank

@for with track and @empty

```
@{{1}} (student of students; {{2}} student.id) { <li>{{ student.name }}</li> } @{{3}} { <li>No students yet</li> }
```

Answers:

- [1] for
- [2] track
- [3] empty

#1265

Midterm

Fill in the Blank

Two-way binding import and usage

```
import { {{1}} } from '@angular/forms';

@Component({
  imports: [ [____2____] ],
  template: `
    <input [____3____]="newTask">
    <button (click)="addTask()">Add</button>
  `
})
```

Answers:

- [1] FormsModule
- [2] FormsModule

#1266

Midterm

Fill in the Blank

Interpolation and expression

```
<h1>Hello {{ {{1}} }}</h1>
```

Answers:

- [1] title

#1267

Midterm

Fill in the Blank

@if and @else

```
@{{1}} (isLoggedIn) { <p>Welcome back!</p> <button (click)="logout()">Logout</button> } @{{2}} { <p>Please log in</p> <button (click)="login()">Login</button> }
```

Answers:

- [1] if
- [2] else

#1268

Midterm

Fill in the Blank

Component class with properties

```
import { {{1}} } from '@angular/core';

@Component({
  [____2____]: 'app-counter',
  templateUrl: './counter.html'
})
export class CounterComponent {
  count = 0;
}
```

Answers:

- [1] Component
- [2] selector

#1269

Midterm

Fill in the Blank

Property binding on element

 <button {{2}}="isDisabled">Submit</button>

#1270

Midterm

Fill in the Blank

Child component with input and output

```
export class TaskItemComponent { task = {{1}}<string>(); index = {{2}}<number>(); delete = {{3}}<number>();
  onDelete() {
    this.delete.[____4____](this.index());
  }
}
```

Answers:

- [1] input
- [2] input
- [3] output
- [4] emit

#1271

Midterm

Fill in the Blank

Parent template using child component

```
@for (task of tasks; track task) { <app-task-item {{1}}="task" {{2}}="$index" {{3}}="deleteTask($event)" /> }
```

#1272

Midterm

Fill in the Blank

Lifecycle hooks implementation

```
export class MyComponent implements OnInit, OnChanges, OnDestroy { {{1}}() { console.log('Component
initialized'); }
  [____2____](changes: SimpleChanges) {
    console.log('Input changed');
  }
  [____3____]() {
    console.log('Component destroyed');
  }
}
```

Answers:

- [1] ngOnInit
- [2] ngOnChanges
- [3] ngOnDestroy

#1273

Midterm

Fill in the Blank

Standalone component imports

```
@Component({ selector: 'app-root', {{1}}: [RouterOutlet, FormsModule, StudentCard], templateUrl: './app.html' })
```

Answers:

- [1] imports

#1274

Midterm

Fill in the Blank

Input signal access in template

```
<div class="student-card"> <h4>{{ student{{1}}.name }}</h4> <p>Email: {{ student{{2}}.email }}</p> </div>
```

Answers:

- [1] ()
- [2] ()

#1275

Midterm

Fill in the Blank

Output event emitter

```
delete = {{1}}<number>();  
onDelete() {  
  this.delete.[__2__](this.student().id);  
}
```

Answers:

- [1] output
- [2] emit

#1276

Midterm

Fill in the Blank

Service with dependency injection

```
import { {{1}}, inject } from '@angular/core'; import { HttpClient } from '@angular/common/http';  
@[__2__]({  
  providedIn: '[__3__]'  
})  
export class PostService {  
  private http = [__4__](HttpClient);  
}
```

Answers:

- [1] Injectable
- [2] Injectable
- [3] root
- [4] inject

#1277

Midterm

Fill in the Blank

HTTP GET with Observable

```
getPosts(): {{1}}<Post[]> { return this.http.{{2}}<Post[]>(`${this.BASE_URL}/posts`); }
```

Answers:

- [1] Observable
- [2] get

#1278

Midterm

Fill in the Blank

Subscribe to Observable

```
ngOnInit() { this.postService.getPosts().{{1}}({ {{2}}: (posts) => { this.posts.set(posts); }, {{3}}: (err) => {  
this.error.set(err); } }); }
```

Answers:

- [1] subscribe
- [2] next
- [3] error

#1279

Midterm

Fill in the Blank

Route with parameter

```
export const routes: Routes = [ { path: '', {{1}}: 'home', pathMatch: 'full' }, { path: 'home', {{2}}: Home }, { path:  
'posts/{{3}}', component: PostDetail }, { path: '{{4}}', component: NotFound } ];
```

Answers:

- [1] redirectTo

- [2] component
- [3] :id
- [4] **

#1280

Midterm

Fill in the Blank

Reading route parameters

```
private route = inject({{1}});

ngOnInit() {
  const id = Number(this.route.[____2____].paramMap.[____3____]('id'));
}
```

Answers:

- [1] ActivatedRoute
- [2] snapshot
- [3] get

#1281

Midterm

Fill in the Blank

Navigation template

```
<nav> <a {{1}}="/home" routerLinkActive="active">Home</a> <a {{2}}="/posts"
routerLinkActive="active">Posts</a> </nav>

<main>
  <[____3____] />
</main>
```

Answers:

- [1] routerLink
- [2] routerLink
- [3] router-outlet

#1282

Midterm

Fill in the Blank

Signal state management

```
posts = {{1}}<Post[]>([]); loading = {{2}}(false); error = {{3}}("");

ngOnInit() {
  this.loading.[____4____](true);
  this.postService.getPosts().subscribe({
    next: (data) => {
      this.posts.[____5____](data);
      this.loading.[____6____](false);
    }
  });
}
```

Answers:

- [1] signal
 - [2] signal
 - [3] signal
 - [4] set
 - [5] set
 - [6] set
-